

R0 — C, do PC ao micro

Binário, hexadecimal, máscaras, e o primeiro código gravado

Microcontroladores — DENE/UFMT

OBJETIVOS

- Escrever a mesma lógica em Python e em C, e reconhecer o que muda quando o destino é um microcontrolador.
- Converter entre binário, hexadecimal e decimal para valores de oito bits, sem calculadora.
- Ligar, apagar, inverter e testar um bit isolado sem alterar os vizinhos.
- Compilar e gravar pelo bootloader, e identificar por sintoma as falhas mais comuns do caminho.
- Produzir um atraso por laço contado e registrar um palpite de período **antes** de medir.

Este roteiro não vale nota, e é a única sessão do semestre em que isso é verdade junto com o R1.

SEM NOTA

Ele existe para que o erro aconteça aqui. Escreva máscaras erradas de propósito, troque a ordem das linhas, grave o arquivo errado. A partir do R2 a previsão registrada antes do experimento vale nota — e ela é avaliada pelo raciocínio, não pelo acerto.

PERIGO

Uma regra, e ela vale o semestre inteiro: **a garra de terra da ponta de prova vai num ponto de GND, e em nenhum outro lugar**. Prendê-la num ponto de 12 V — LAMP, HEATER, COOLER — curto-circuita 12 V contra o terra da rede.

Hoje você não usa osciloscópio nem nenhum desses pontos. O aviso vem agora porque o hábito se forma antes de haver risco, e porque, cumprida essa regra, medir é seguro — o que importa muito neste curso.

Configuração de bancada para esta sessão:

NESTE KIT

Chaves SWITCHS (PORTB)	Todas em OFF
CH2-1 (LCD)	OFF
CH3-3 e CH3-5	OFF
Demais bancos	OFF

Os únicos periféricos usados hoje são os oito LEDs.

1 A mesma lógica, três vezes

Vocês já escreveram Python. O mesmo raciocínio, escrito de três formas, mostra o que a mudança de destino custa.

```
# Python, no computador
ligado = False
while True:
    ligado = not ligado
    print("LED", "aceso" if ligado else "apagado")
    time.sleep(0.5)
```

```
/* C, no computador */
#include <stdio.h>
int main(void) {
    int ligado = 0;
```

```

while (1) {
    ligado = !ligado;
    printf("LED %s\n", ligado ? "aceso" : "apagado");
    /* dorme meio segundo */
}
}

/* C, no microcontrolador */
#include <xc.h>
#include <stdint.h>
void main(void) {
    LATD = 0xFF;          /* estado antes de virar saída */
    TRISD = 0x00;         /* agora sim: saída */
    while (1) {
        LATD ^= 0b00000001; /* inverte o bit 0 */
        for (uint16_t i = 0; i < 30000u; i++) { } /* atraso */
    }
}

```

Três diferenças, e nenhuma é de sintaxe.

CONCEITO

Não há para onde retornar. `main` não termina. Se o laço acabar, o processador continua buscando instruções no que vier depois — que não é código seu.

Não há `print`. A saída é um pino. E escrever num pino é escrever numa posição de memória: `LATD` é o nome de um endereço, e o encontro 1 mostra qual.

Não há `sleep`. Ninguém está esperando por você. O atraso é um laço que gasta instruções, e o tempo que ele consome é consequência — não escolha.

Tarefa 1. Antes de gravar qualquer coisa, responda no caderno: quantas vezes o laço `for` executa? Quantas instruções você acha que cada volta custa? Que período isso dá?

TAREFA

Não há como acertar hoje — o número de instruções por volta ainda não foi apresentado. **Registre o palpite e a base dele.** O encontro 2 devolve a conta, e o R3 mede.

2 Bases: por que hexadecimal

Um registrador de oito bits tem 256 estados. Escrevê-los em decimal esconde exatamente a informação que interessa: **quais bits estão em 1**.

Binário	Hexa	Decimal	Leitura
0b00000001	0x01	1	só o bit 0
0b00001000	0x08	8	só o bit 3
0b10000000	0x80	128	só o bit 7
0b00001111	0x0F	15	os quatro bits baixos
0b11110000	0xF0	240	os quatro bits altos
0b11111111	0xFF	255	todos

Cada dígito hexadecimal vale exatamente quatro bits, o que torna a conversão mecânica: parta o byte em dois grupos de quatro e traduza cada um.

Tarefa 2. Complete sem calculadora, e confira com o colega antes de conferir com a máquina.

TAREFA

Binário	Hexa	Decimal
0b00100000		
	0x24	
		192
0b01010101		
	0xC3	

2.1 Largura explícita

O tamanho de `int` não é definido pela linguagem C: varia entre plataformas. Código escrito com `int` funciona numa arquitetura e falha silenciosamente em outra.

ATENÇÃO

Tipo	Faixa	Uso típico neste curso
<code>uint8_t</code>	0 a 255	Registrador, máscara, contador curto
<code>uint16_t</code>	0 a 65535	Leitura do conversor, pré-carga de temporizador
<code>int16_t</code>	– 32768 a 32767	Temperatura em décimos de grau

Há uma segunda razão, desta plataforma: são 2 kB de RAM. Um valor de 16 bits onde bastariam 8 desperdiça metade do espaço, e o desperdício só aparece no dia em que o programa deixa de caber — quando já é caro corrigir.

NOTA

E uma terceira: o compilador na versão gratuita não oferece ponto flutuante utilizável aqui. Por isso a temperatura vai ser guardada em **décimos de grau** como `int16_t`, e não como `float`. Restrição de ferramenta virando decisão de projeto, padrão que vai se repetir o semestre inteiro.

3 Os seis operadores

Operador	Nome	Para que serve, na prática
<code>&</code>	E bit a bit	Zerar bits; testar se um bit está em 1
<code> </code>	OU bit a bit	Pôr bits em 1 sem tocar nos demais
<code>^</code>	OU-exclusivo	Inverter bits selecionados
<code>~</code>	Complemento	Transformar uma máscara no seu negativo
<code><<</code>	Deslocamento à esquerda	Construir a máscara do bit n
<code>>></code>	Deslocamento à direita	Trazer um campo para os bits baixos

Não confunda `&` com `&&`, nem `|` com `||`. Os dobrados são **lógicos**: trabalham com verdadeiro e falso e devolvem 0 ou 1. Os simples trabalham bit a bit.

ATENÇÃO

`PORTB && 0x04` quase nunca é o que você quis escrever, e o compilador não reclama.

3.1 As quatro operações que resolvem o semestre

```
#include <stdint.h>

#define BIT(n) (1u << (n))

uint8_t r = 0b00000000;

r |= BIT(2);          /* liga o bit 2, preserva o resto */
r &= ~BIT(2);         /* apaga o bit 2, preserva o resto */
r ^= BIT(2);          /* inverte o bit 2 */

if (r & BIT(2)) {     /* testa: verdadeiro se o bit 2 e 1 */
}
```

Reconheça o padrão, em vez de decorar as linhas: **ligar é OU com a máscara; apagar é E com a máscara invertida; inverter é OU-exclusivo; testar é E seguido de comparação.**

CONCEITO

O ~ do apagar é o erro mais comum de todos. Sem ele, `r &= BIT(2)` zera todos os outros bits em vez de preservar — e o sintoma, “mexi num e sumiram os outros”, é inconfundível depois que se vê uma vez.

Tarefa 3. Complete, escrevendo binário e hexadecimal.

TAREFA

Expressão	Binário	Hexa
BIT(0)	_____	_____
BIT(3)	_____	_____
BIT(7)	_____	_____
BIT(2) BIT(5)	_____	_____
~BIT(0)	_____	_____
~(BIT(1) BIT(6))	_____	_____

Tarefa 4. Partindo de `uint8_t r = 0b10110100;`, escreva o valor de `r` depois de cada linha, **em sequência** — cada uma opera sobre o resultado da anterior.

TAREFA

1. `r |= BIT(0);` _____
2. `r &= ~BIT(4);` _____
3. `r ^= 0b00001111;` _____
4. `r = BIT(1);` _____

A quarta é a pegadinha. Explique numa frase por que ela é diferente das três anteriores.

4 Do arquivo ao chip

Existem dois caminhos para pôr código dentro do chip, e este curso usa os dois em momentos diferentes.

CONCEITO

Bootloader. Um programa que já está gravado na Flash e que, ao ser acionado, escuta a USB e regrava o resto da memória. É o caminho de vocês. Não precisa de hardware nenhum além do cabo.

Gravador. Um aparelho que fala com o chip por cinco fios e escreve na Flash diretamente, inclusive os bits de configuração. É o caminho da demonstração do encontro 0, e o único possível num chip virgem.

Para entrar em modo bootloader: segure **SW9** e energize a placa.

NESTE KIT

É SW9, e não SW1. O manual da Exsto diz SW1 e está errado — a serigrafia da placa tem precedência sobre o manual.

Tarefa 5. Crie o projeto no ambiente, selecione o PIC18F4550, compile o programa de três linhas da §1 e grave pelo bootloader.

TAREFA

Anote o tempo total do ciclo: da edição do arquivo até o LED responder. Esse número vai importar quando o programa tiver quatrocentas linhas.

Não escreva #pragma config neste curso. Os bits de configuração pertencem ao bootloader, que já os gravou. O que você escrever ali é **silenciosamente ignorado** — não dá erro, não dá aviso, e não tem efeito.

ATENÇÃO

É a primeira aparição do princípio da §4.3 da aula 0: existe código que rodou antes do seu, e ele é dono de coisas.

4.1 Falhas por sintoma

Sintoma	O que costuma ser
undeclared identifier 'uint8_t'	Faltou <code>#include <stdint.h></code>
'LATD' undeclared	Faltou <code>#include <xc.h></code> , ou o dispositivo do projeto está errado
expected ';'	O ponto-e-vírgula que falta está na linha anterior
O gravador não encontra a placa	Não entrou em modo bootloader: SW9 pressionado antes de energizar
Compila, grava, e nada muda	Gravou um arquivo antigo — confira a data do .hex

Leia sempre a **primeira** mensagem de erro, não a última. O compilador perde o fio depois do primeiro tropeço e passa a reclamar de linhas que estão perfeitas. Corrigir a primeira costuma apagar dez.

NOTA

5 O byte que você escreve e o painel que responde

Tarefa 6. Grave um programa que escreva um valor fixo na porta dos LEDs, com LAT antes de TRIS, começando por `0b11110000`.

TAREFA

1. Desenhe os oito círculos no caderno e marque quais LEDs você **espera** que acendam.
2. Grave e compare.

O resultado provavelmente não foi o que você previu.

EXPERIMENTO

Não tente explicar agora, e não peça a explicação. Registre com precisão: qual byte você escreveu e quais LEDs acenderam. Depois formule uma hipótese sobre o **circuito** — não sobre o seu código — que explique a diferença.

O R1, na semana que vem, começa exatamente por aí. Chegar lá com a hipótese escrita muda o que aquela sessão vale para você.

Tarefa 7. Agora troque a ordem: TRISD antes de LATD. O painel muda?

TAREFA

Se não mudou, isso **prova** que a ordem não importa? Responda em uma frase, e guarde a resposta: o encontro 2 volta a ela com um osciloscópio.

6 Registro

Entrega. Leve o caderno com: as três tabelas de conversão preenchidas, o palpite de período da Tarefa 1 com a base dele, o desenho dos LEDs esperados contra os observados, e a hipótese sobre o circuito.

TAREFA

Devolva a bancada ao estado da tabela de configuração.

Para o R1. Traga escrito, já sabendo que no XM118 RC1 aciona a resistência de aquecimento, RC2 aciona a ventoinha, e RC6 e RC7 são a comunicação serial:

TAREFA

um colega precisa ligar a ventoinha e escreve `LATC = 0b00000100`; . O que acontece com a comunicação serial? E com a resistência?

No R1: o kit XM118 por dentro. Componentes, serigrafia, medidas com multímetro e osciloscópio, e as chaves como entrada de bits.

NOTA

E a resposta da sua hipótese — por que o painel não fez o que o byte dizia.